Rapport TP TMI:
Etude des contours

Thomas Wieczorek Léo Loi

February 10, 2026

1 Calcul des tangentes selon l'estimateur λ-MST

1.1 Calcul de la couverture tangentielle

Voici le pseudo-code de l'algorithme utilisé:

$start \leftarrow 0$; ▷ indice du point de départ
$max \leftarrow start + p.getContourTPLgr(start)$; ▷ indice du point le plus éloigné sur la même droite que $start$
$contour \leftarrow []$; #liste des segments maximaux
for $i \leftarrow 0, i < n, i \leftarrow i + 1$ **do**
  if $i + p.getContourTPLgr(i) \neq max$ **then** ▷ nouvelle droite
    $contour.append([start, max\%n])$; ▷ On ajoute le segment aux segments maximaux
    $start \leftarrow i$;
    $max \leftarrow start + p.getContourTPLgr(start)$;
  end if
end for
if $max\%n = contour[0][1]$ **then** ▷ Cas d'une boucle (dernier start appartient au premier segment)
  $contour[0][0] \leftarrow start$;
else
  $contour.append([start, max\%n])$
end if

En appliquant cet algorithme à l'image $testforme.pgm$, nous avons obtenu les segments maximaux suivants :
Segment 1 : 48 → 19
Segment 2 : 15 → 20
Segment 3 : 19 → 24
Segment 4 : 20 → 30
Segment 5 : 26 → 40
Segment 6 : 36 → 45
Segment 7 : 37 → 46
Segment 8 : 44 → 49
Segment 9 : 45 → 0
Segment 10 : 46 → 1

1.2 Calcul des tangentes selon l'estimateur λ-MST

Description des calculs et résultats des points suivants:

- Point d'indice 20 :
  $e_{MS}(i)$:
  - Segment 2 : 0
  - Segment 3 : 0.2
  - Segment 4 : 0

  Total = 0.2

  θ_{MS} :
  - Segment 2 : -2.55359
  - Segment 3 : -0.588003
  - Segment 4 : -0.244979

  Total =

  $\hat{\theta}(i)$ = -0.588003
- Point d'indice 48
  $e_{MS}(i)$:
  - Segment 1 : 0
  - Segment 8 : 0.2
  - Segment 9 : 0.4
  - Segment 10 : 0.4

  Total = 1

  θ_{MS} :
  - Segment 1 : -2.83871
  - Segment 8 : 2.1588
  - Segment 9 : 2.55359
  - Segment 10 : 2.89661

  Total =

  $\hat{\theta}(i)$ = 2.61184
- Point d'indice 49
  $e_{MS}(i)$:
  - Segment 1 : 0.047619
  - Segment 8 : 0
  - Segment 9 : 0.2

- Segment 10 : 0.4

Total = 0.2

θ_{MS} :

- Segment 1 : -2.83871
- Segment 8 : 2.1588
- Segment 9 : 2.55359
- Segment 10 : 2.89661

Total =

$\hat{\theta}(i) = 2.36897$

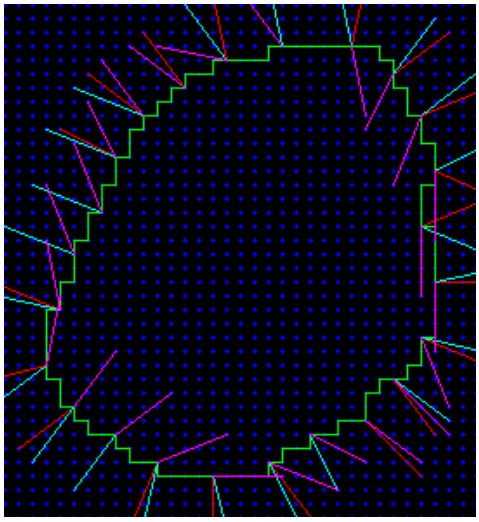


Figure 1: Normales aux tangentes λ-MST de l'image *blob.pgm*

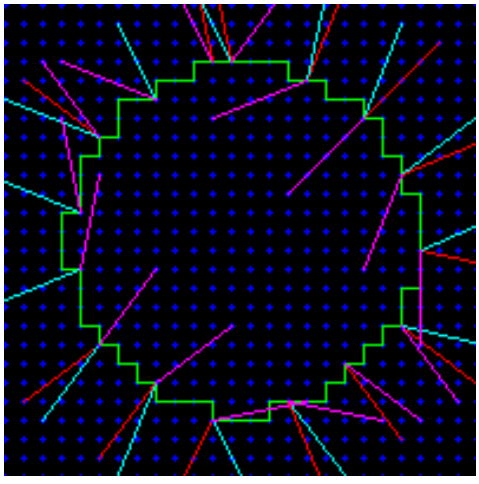


Figure 2: Normales aux tangentes λ-MST de l'image *disque.pgm*

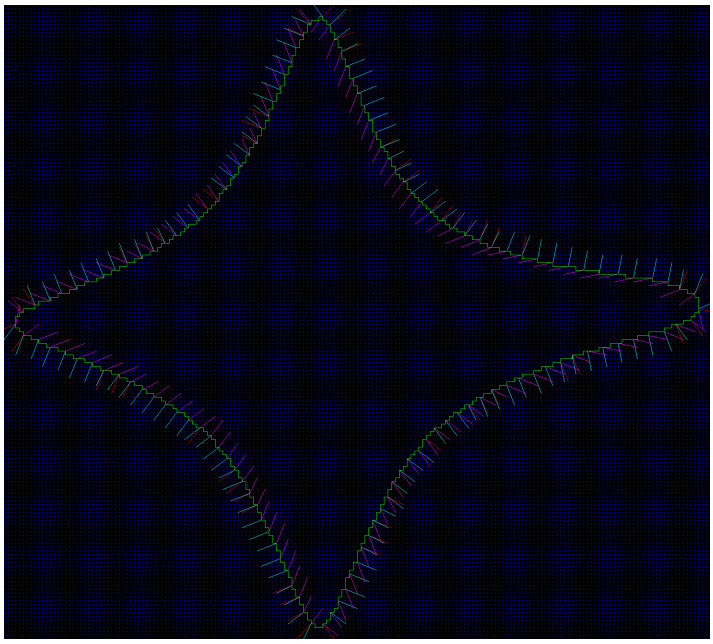


Figure 3: Normales aux tangentes λ-MST de l'image $etoile1.pgm$

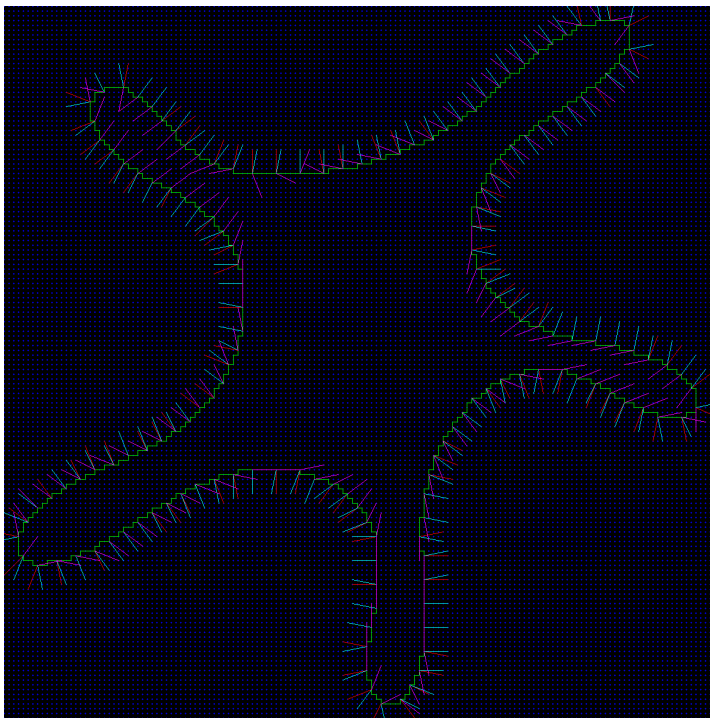


Figure 4: Normales aux tangentes λ-MST de l'image $etoile2.pgm$

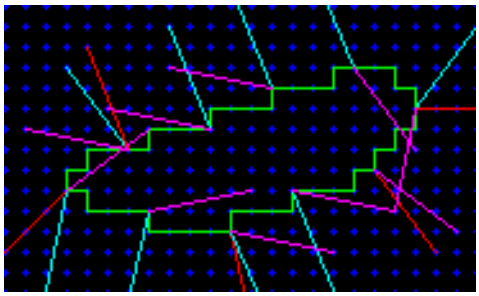


Figure 5: Normales aux tangentes λ-MST de l'image $formeTest.pgm$

1.3 Étude des deux publications

Afin de comparer les différents estimateurs de tangente, les deux publications procède de manière différente. Dans la première (J.-O. Lachaud, A. Vialard, F. de Vieilleville, Fast, accurate and convergent tangent estimation on digital contours, in Image and Vision Computing, vol 25, 2007.), les chercheurs ont choisit 3 formes (cercle, fleur et rsquar) sur lesquelles ils ont fait fonctionner les 4 algorithmes à des pas différents. Ils ont ensuit comparer 3 valeurs :

- l'erreur absolue moyenne
- l'erreur absolue maximale
- l'écart type de l'erreur absolue moyenne

Ils en conclut ici que la λ-MST semble être lestimateur le plus robuste et le souvent le plus précis.

Dans la deuxième publication (F. de Vieilleville, J.-O. Lachaud, Fast, accurate and convergent tangent estimation on digital contours Comparison and improvement of tangent estimators on digital curves, in Pattern Recognition, vol 42, 2009), les chercheurs ont comparés des estimateurs avec des méthodes continues dans le cas idéal de digitalisation. Ils ont notamment comparé leur temps d'exécution et leur précision. Dans cette étude, l'estimateur qui semble ressortir comme le plus précis et le moins coûteux en temps est le GMC suivit par le λ-MST.

Dans notre cas, il est difficile de juger, notre code ne devant pas être très bon car les normales aux tangentes ne sont pas toutes orientées dans le même sens (extérieur de la forme). Cependant, en essayant de tracer les tangentes visuellement, les normales crées par λ-MST ont l'air plus précise.

2 Calcul de la longueur des contours

L'algorithme suivant a été utilisé pour déterminer la polygonisation à l'aide des extrémités des tangentes positives

```
contour <- CodeDeFreeman()
P <- []
seuil <- 0 #a definir
pour num_point dans contour faire
    si distance(tangente_positive1,tangente_positive2) > seuil  alors
```

```
        ajouter num_point à P
finfaire
pour num_point dans P faire
    trouver num_point et num_point+1 dans le contour
    dessiner le segment entre les deux
finfaire
trouver et dessiner le segment entre le dernier et le premier point
```

Avec un seuil de 2 les polygones suivants ont été obtenus:



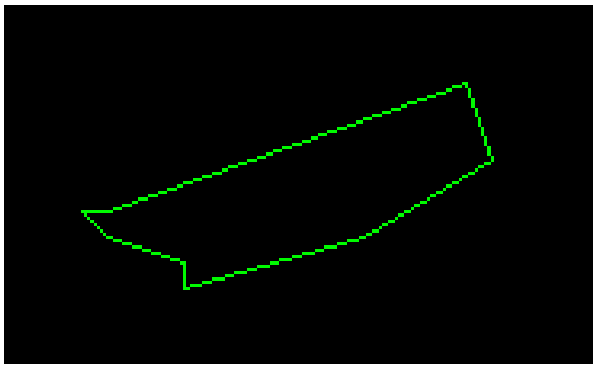


Figure 6: formeTest: longueur estimée: 299

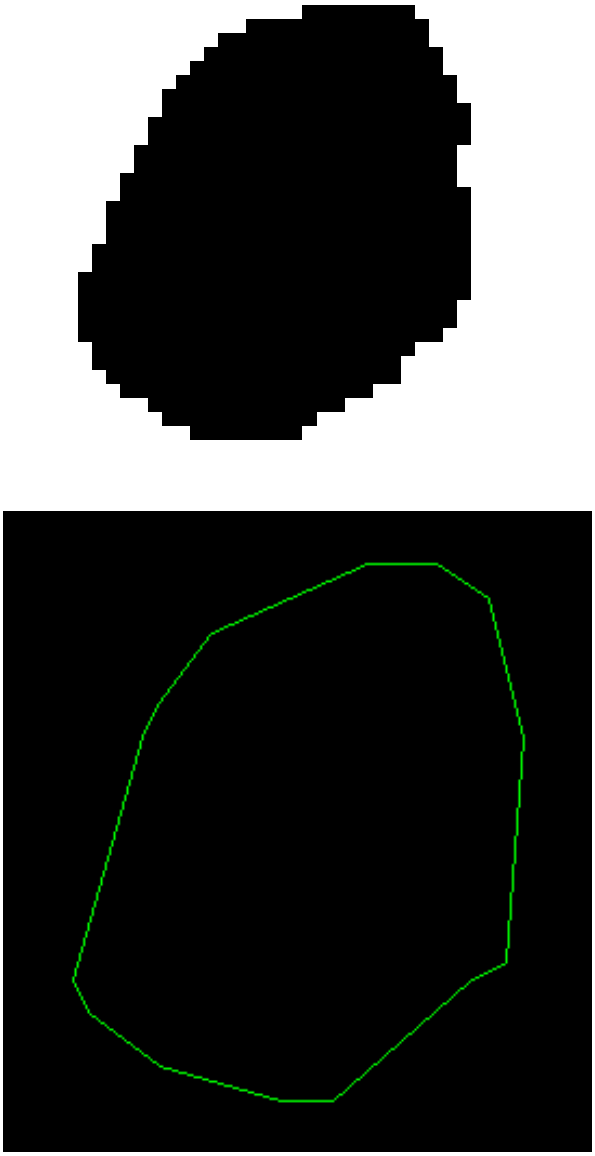


Figure 7: blob: longueur estimée: 721

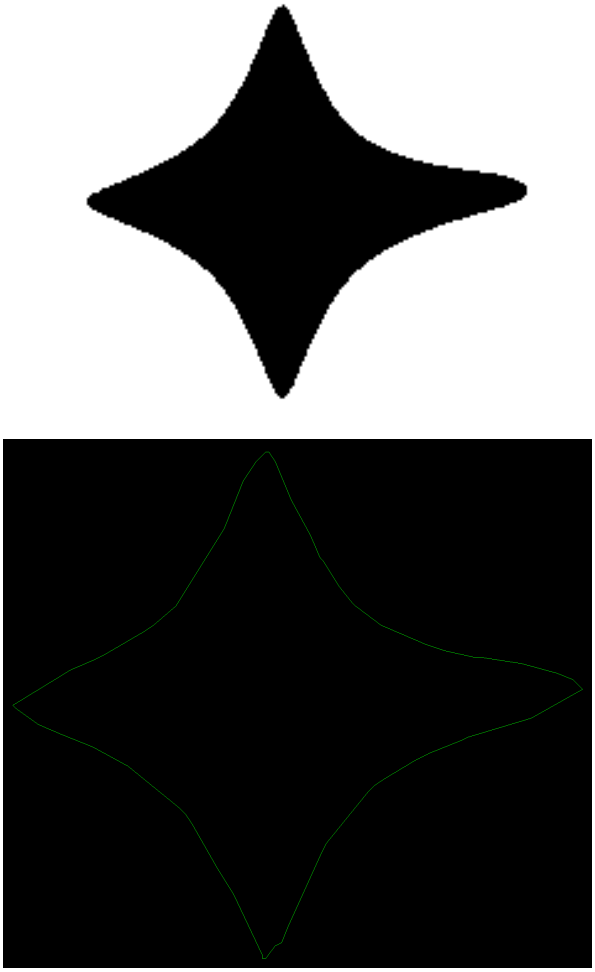


Figure 8: Etoile1: Longueur estimée: 3990

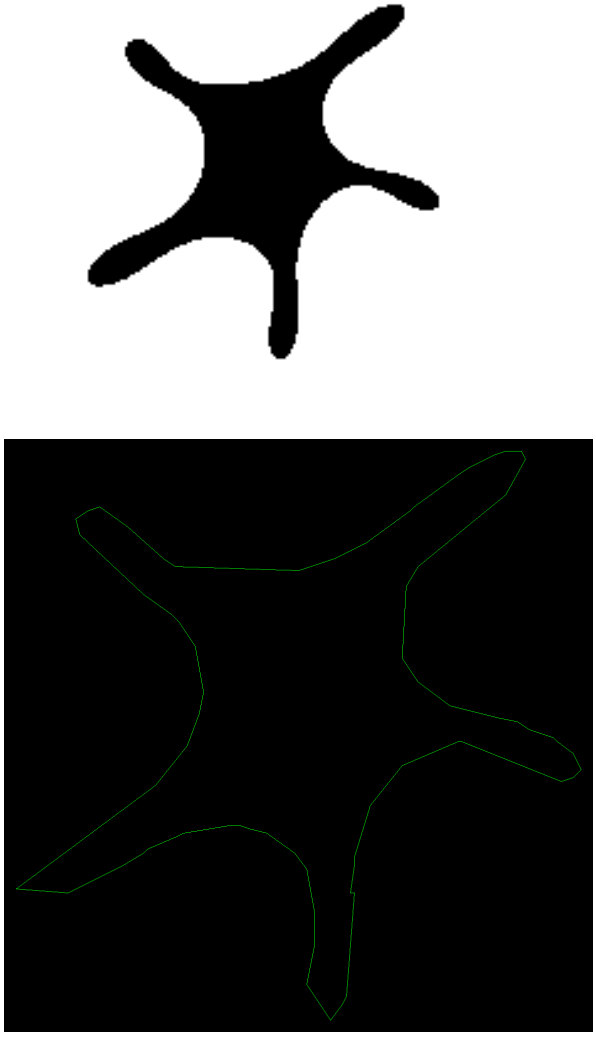


Figure 9: Etoile2: longueur estimée: 4875

Pour le disque un seuil de 1 était plus satisfaisant

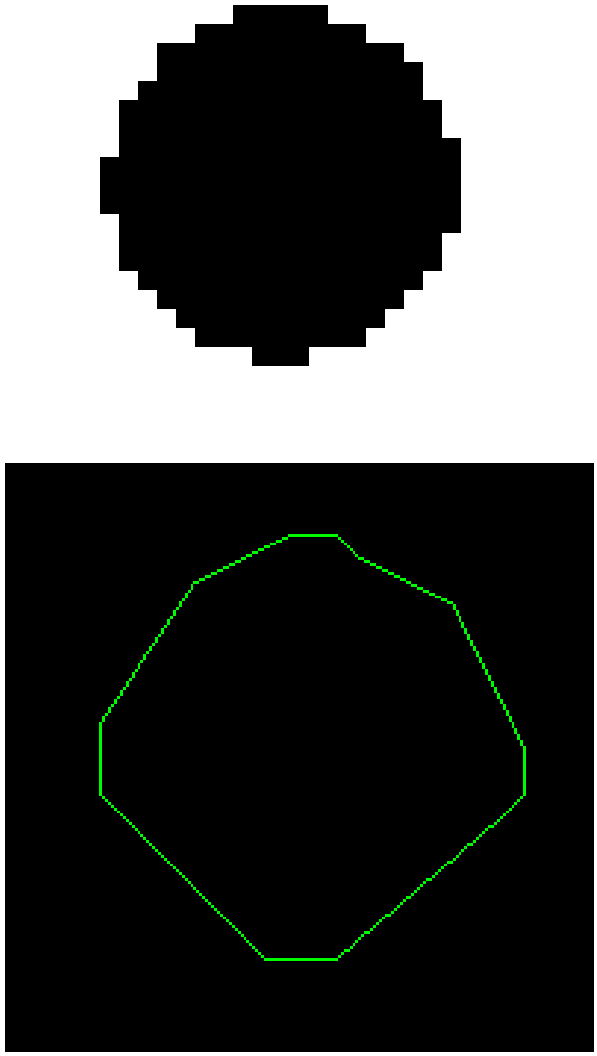


Figure 10: Disque: longueur estimée: 432

3 Calcul de la courbure

Pour calculer la courbure la méthode des demi-tangentes a été utilisée, avec un m de 5

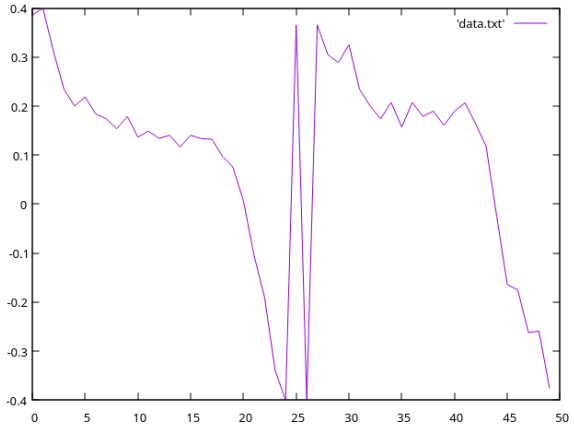


Figure 11: Courbure de formeTest

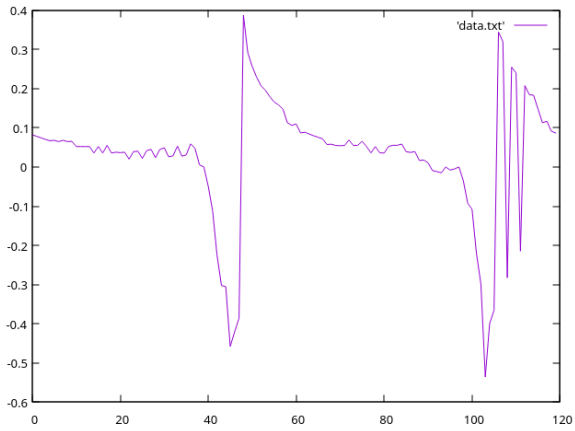


Figure 12: Courbure de blob

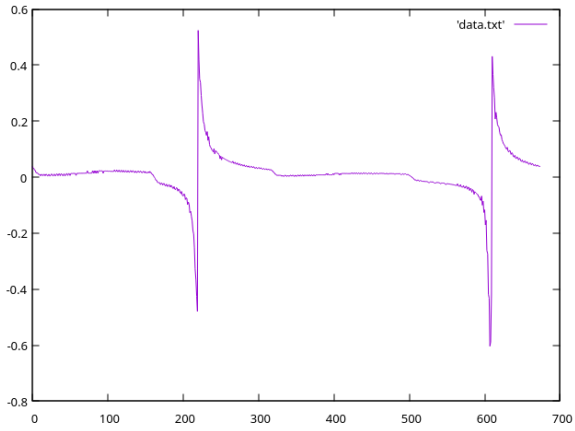


Figure 13: Courbure de etoile1

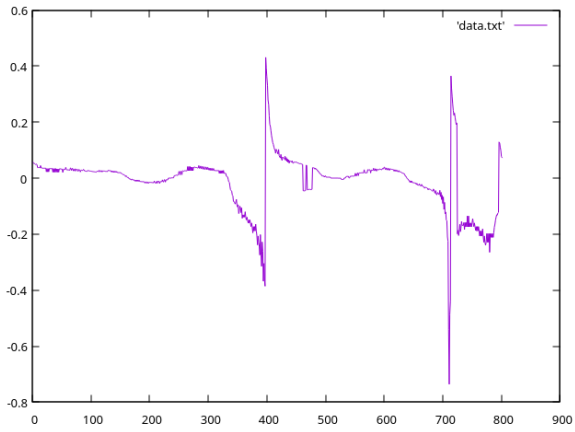


Figure 14: Courbure de etoile2

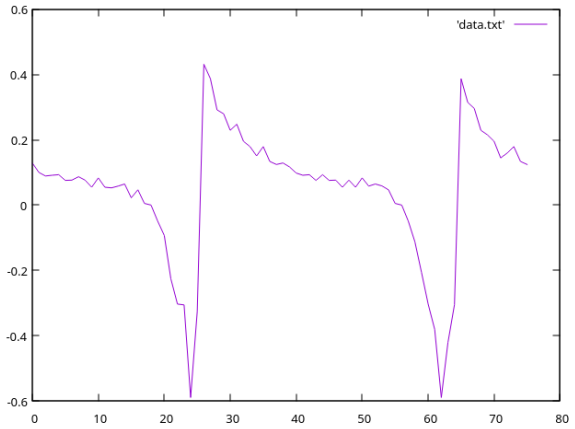


Figure 15: courbure de disque

Ces courbes permettent de voir les grandes différences de courbure et donc de pouvoir avoir une idée de la forme de l'image, par exemple les "spikes" représentent un passage vers une pointe et le rapprochement des patterns nous donne la proximité dans l'image

Cela se voit moins sur les fichiers étoile mais ces fichiers sont beaucoup plus conséquents et ont beaucoup plus de points (le paramètre m à 5 est bien trop faible pour repérer leurs changements de courbure)

Pour rajouter des formes de test on pourrait ajouter des formes avec des zones plus régulières (une croix par exemple), d'autres plus complexes

On pourrait également ajouter des images avec des zones convexes et concaves bien définies (un peu comme les étoiles) ou peut être des flèches pour observer les changements de courbure.